Real time systems

My project is an automatic moisture detection robot with remote control capabilities. its function is to automatically map out the underside of a house and find and record points of excess moisture with an IR camera and GPS. The goal is to prevent catastrophic water damage before it can worsen.

The components list:

1. 10,000 Mah Battery
2. Raspberry pi 4b
3. Arduino nano IoT
4. Motor controller unit
5. 4x motors
6. 2x ultra sonic sensors
7. GPS module
8. Soil moisture sensor
9. Humidity sensor
10. IR camera
11. Robot Chassis

the soft requirements:

The main soft requirements apply to the IR camera data collection and GPS logging. a delay in processing an image or updating a coordinate does not result in any catastrophic damage but will still give a less accurate moisture map.

The hard requirements:

The main hard requirements are the obstacle avoidance system i.e. The ultrasonic sensors: if the system fails to process a "stop" command before physical contact, the buggy could become stuck and hardware damage may occur.

Approaches to Achieve Real-Time Response

- **Approach 1: Hardware Interrupts (Arduino Side):** To ensure the hard real-time requirement of obstacle avoidance is met, I will use hardware interrupts on the Arduino Nano for the ultra-sonic sensor. By attaching an interrupt to the Echo pin of the ultrasonic sensor, the processor can measure the pulse duration instantly without being blocked by other code, such as the moisture sensor readings and GPS.
- **Approach 2: Multi-threading/Asynchronous Processing (Raspberry Pi Side):** On the Raspberry Pi, which runs a non-deterministic OS (Linux), I will use multi-threading to separate the IR camera feed from the navigation logic. This ensures that a heavy image-processing task doesn't "freeze" the command link to the motors, maintaining system responsiveness.

Justification:

The use of Interrupts is beneficial because it provides a deterministic response. In the narrow, dark environment of a sub-floor, the buggy must react to obstacles with precision. If we used "polling" a slow moisture sensor read coupled with the long GPS readings could delay the obstacle check just long enough for the buggy to hit something. Interrupts ensure that safety-critical tasks always take priority over data-collection tasks.

The use of multithreading will allow everything to run simultaneously without any need for interrupts and minimise delays when it comes to motor control.